

MLOps Practical – 3

Kubeflow Pipelines Backend, Manual Execution & Scheduling

Complete Student-Friendly Theory + Hands-on Laboratory Manual

Item	Details
Course	MLOps
Unit	Model Deployment & Pipeline Orchestration
Practical	3
Technology	Kubeflow Pipelines 2.17.0 + Kubernetes
ML Example	Iris classification + quality gate
Schedule demonstration	Every 3 minutes
Execution	KFP Backend on Kubernetes

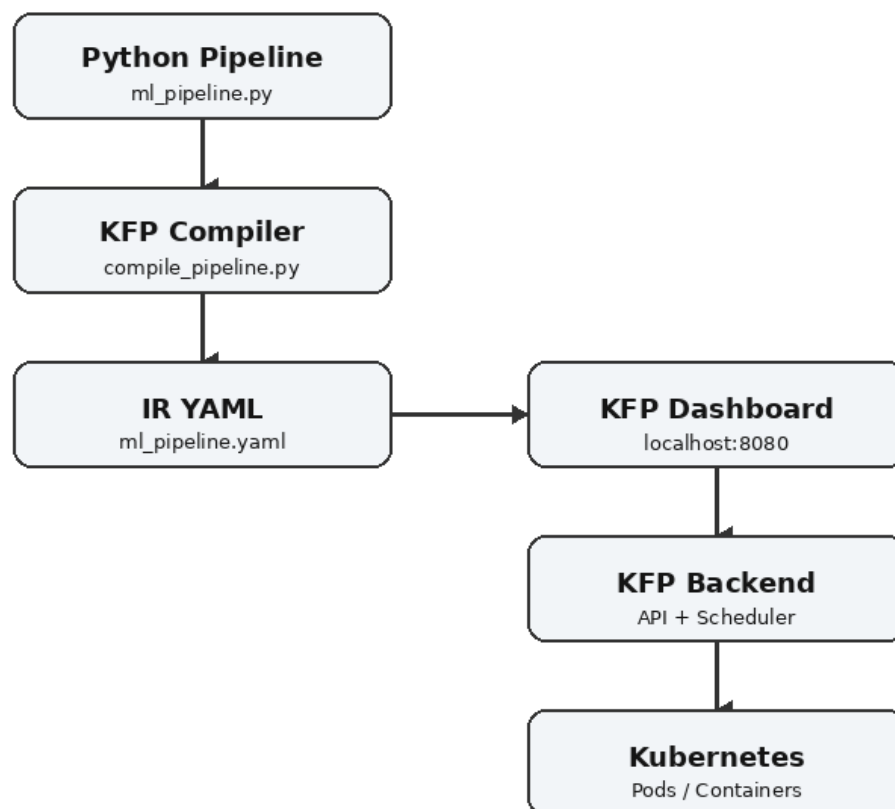
What students will build: A complete KFP workflow that is compiled, uploaded, executed manually, and then automatically executed by a recurring scheduler.

Contents

1. 1. Practical Overview
2. 2. Objectives and Learning Outcomes
3. 3. Prerequisites and Environment
4. 4. Theory: KFP Backend
5. 5. Local KFP vs Backend KFP
6. 6. Architecture and Components
7. 7. Pipeline Used in the Practical
8. 8. Project Files and Code
9. 9. Hands-on Part A: Verify Environment
10. 10. Hands-on Part B: Compile Pipeline
11. 11. Hands-on Part C: Start KFP Dashboard
12. 12. Hands-on Part D: Upload Pipeline
13. 13. Hands-on Part E: Manual Run
14. 14. Hands-on Part F: Create Recurring Run
15. 15. Hands-on Part G: Verify Automatic Runs
16. 16. Detailed Explanation of Scheduling
17. 17. Error Tracing and Troubleshooting
18. 18. Observed Lab Errors and Their Resolution
19. 19. Practical Result and Evidence
20. 20. Viva Questions
21. 21. Student Submission Checklist
22. 22. Quick Command Sheet
23. 23. Related Files and How to Use Them
24. 24. Official References

1. Practical Overview

This practical takes the Iris ML pipeline from local development into a KFP backend. Students first understand the pipeline, compile it into an intermediate-representation YAML file, upload it to the KFP Dashboard, execute it manually, and finally configure a recurring run that automatically starts a new pipeline execution every three minutes.



Development → Compile → Backend → Kubernetes

Simple idea: Python defines the workflow → the compiler creates YAML → KFP Backend manages it → Kubernetes executes the tasks → the scheduler starts new runs automatically.

2. Objectives and Learning Outcomes

Objectives:

- Understand the role of the KFP backend in MLOps.
- Verify a Kubernetes-based KFP installation.
- Compile a KFP pipeline to IR YAML.
- Upload and run a pipeline through the KFP Dashboard.
- Read task status and logs.
- Create a recurring run.
- Configure a Periodic trigger for every 3 minutes.
- Trace common UI, Kubernetes, and scheduler errors.

Learning outcomes:

- Explain KFP Backend vs local execution.
- Compile and upload a pipeline.
- Execute and monitor a pipeline.
- Configure and verify automatic scheduling.
- Identify the likely cause of a broken port-forward.
- Inspect KFP and Kubernetes logs for failures.

3. Prerequisites and Environment

Item	Lab value / purpose
Windows	Windows 10/11 host
WSL2	Linux environment
Ubuntu	Command-line environment
Docker Desktop	Container runtime and Kubernetes integration
Kubernetes	Execution platform
kubectl	Kubernetes CLI
Python	Pipeline development
KFP SDK	Pipeline definition and compilation
KFP Backend	Pipeline management, execution and scheduling
KFP version	2.17.0 in this lab
Python environment	Project virtual environment; previously verified with Python 3.14.4

The practical assumes Docker Desktop, WSL2, Kubernetes, and the KFP backend are already installed. Installation/reinstallation of the backend is documented in troubleshooting notes; the main focus is backend use and scheduling.

4. Theory: What is the KFP Backend?

Kubeflow Pipelines (KFP) provides a way to define ML workflows as connected components. The backend receives a compiled pipeline specification, stores and manages pipeline/run information, and coordinates execution in the Kubernetes environment.

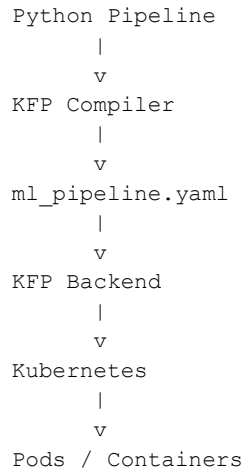
4.1 Local execution

```

Python Pipeline
  |
  v
KFP Local Runner
  |
  v
Local machine
  |
  v
Tasks execute locally

```

4.2 Backend execution

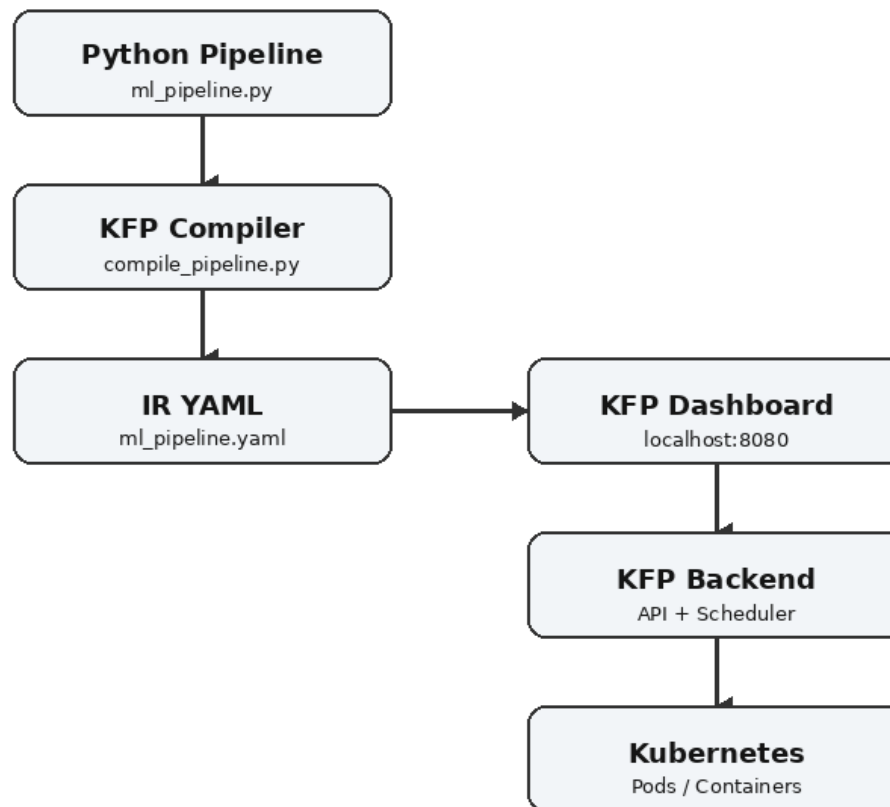


Student definition: Local execution is mainly for testing. Backend execution adds centralized management, dashboard monitoring, run history and recurring scheduling.

5. Local KFP vs Backend KFP

Feature	Local execution	KFP Backend
Execution	Local machine	Kubernetes
Dashboard	No central dashboard	Yes
Run history	Local/testing oriented	Centralized
Recurring scheduling	Not the focus	Yes
Task orchestration	Local runner	Backend + Kubernetes
Monitoring	Terminal/local output	Dashboard + logs
Best use	Development/testing	Managed workflow orchestration

6. Architecture and Components



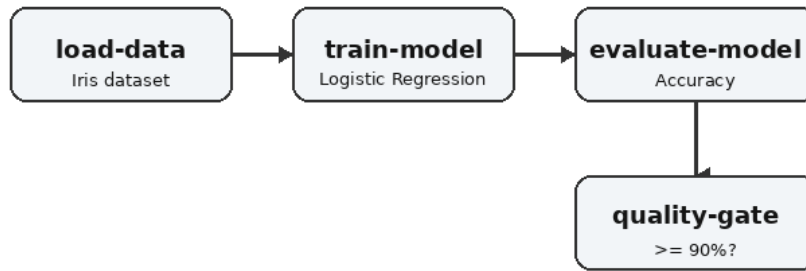
Development → Compile → Backend → Kubernetes

Component	Student-friendly role
ml-pipeline	Main KFP backend/API service
ml-pipeline-ui	Web dashboard
ml-pipeline-scheduledworkflow	Scheduling/recurring workflow support
workflow-controller	Coordinates workflow execution
metadata-grpc	Metadata service
mysql	Backend database
cache-server / cache-deployer	Pipeline cache-related services
seaweedfs	Artifact/storage-related service in this deployment

```
kubectl get pods -n kubeflow
```

7. Pipeline Used in the Practical

The Iris classification pipeline has four components:



Pipeline: load-data → train-model → evaluate-model → quality-gate

Scheduler: Recurring Run → Periodic trigger → Every 3 minutes → New Run

Component	What it does
load-data	Loads Iris data, performs a stratified 80/20 train/test split and returns JSON.
train-model	Trains Logistic Regression and serializes the model as base64-encoded pickle.
evaluate-model	Restores the model, predicts test data and returns accuracy.
quality-gate	Checks whether accuracy is at least 0.90; otherwise raises an error.

Observed successful execution:

```

Total samples: 150
Training samples: 120
Testing samples: 30
Model Accuracy: 0.9667
Model Accuracy: 96.67%
QUALITY GATE: PASS
  
```

Why use a quality gate?: It demonstrates that an ML workflow can automatically stop when a model metric fails a defined requirement.

8. Project Files and Code

File	Purpose
ml_pipeline.py	Complete pipeline definition used in this lab.
compile_pipeline.py	Compiles ml_pipeline.py into ml_pipeline.yaml.
requirements.txt	Pins KFP and Docker SDK versions used in the environment.
verify_backend.sh	Checks Kubernetes nodes, KFP pods, services and workflows.
start_kfp_ui.sh	Starts the KFP UI port-forward.
.gitignore	Ignores virtual environments, local outputs and logs.
ml_pipeline.yaml.README	Explains why the real compiled YAML must be generated locally.

8.1 Complete ml_pipeline.py

```

from kfp import dsl
from kfp import local

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["scikit-learn"]
)
def load_data() -> str:

    from sklearn.datasets import load_iris
    from sklearn.model_selection import train_test_split
    import json

    iris = load_iris()

    X_train, X_test, y_train, y_test = train_test_split(
        iris.data,
        iris.target,
        test_size=0.2,
        random_state=42,
        stratify=iris.target
    )

    print(f"Total samples: {len(iris.data)}")
    print(f"Training samples: {len(X_train)}")
    print(f"Testing samples: {len(X_test)}")

    data = {
        "X_train": X_train.tolist(),
        "X_test": X_test.tolist(),
        "y_train": y_train.tolist(),
        "y_test": y_test.tolist()
    }

    return json.dumps(data)

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["scikit-learn"]
)
def train_model(data_json: str) -> str:

    import json
    import pickle
    import base64
    from sklearn.linear_model import LogisticRegression

    data = json.loads(data_json)

    X_train = data["X_train"]
    y_train = data["y_train"]

    model = LogisticRegression(max_iter=200)
    model.fit(X_train, y_train)

    model_bytes = pickle.dumps(model)
    model_encoded = base64.b64encode(model_bytes).decode("utf-8")

    print("Model training completed.")

    return model_encoded

```

```

@dsl.component(
    base_image="python:3.11",
    packages_to_install=["scikit-learn"]
)
def evaluate_model(
    model_encoded: str,
    data_json: str
) -> float:

    import json
    import pickle
    import base64
    from sklearn.metrics import accuracy_score

    data = json.loads(data_json)

    X_test = data["X_test"]
    y_test = data["y_test"]

    model_bytes = base64.b64decode(model_encoded)
    model = pickle.loads(model_bytes)

    predictions = model.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)

    print(f"Model Accuracy: {accuracy:.4f}")
    print(f"Model Accuracy: {accuracy * 100:.2f}%")

    return accuracy

@dsl.component
def quality_gate(
    accuracy: float,
    threshold: float = 0.90
):

    print(f"Required accuracy: {threshold * 100:.2f}%")
    print(f"Actual accuracy: {accuracy * 100:.2f}%")

    if accuracy >= threshold:
        print("QUALITY GATE: PASS")
    else:
        raise RuntimeError(
            f"QUALITY GATE: FAIL - "
            f"Accuracy {accuracy:.4f} is below "
            f"required threshold {threshold:.4f}"
        )

@dsl.pipeline
def ml_pipeline():

    data_task = load_data()

    train_task = train_model(
        data_json=data_task.output
    )

    evaluate_task = evaluate_model(

```

```

        model_encoded=train_task.output,
        data_json=data_task.output
    )

    quality_gate(
        accuracy=evaluate_task.output,
        threshold=0.90
    )

if __name__ == "__main__":

    local.init(
        runner=local.SubprocessRunner()
    )

    ml_pipeline()

```

8.2 Complete compile_pipeline.py

```

from kfp import compiler
from ml_pipeline import ml_pipeline

compiler.Compiler().compile(
    pipeline_func=ml_pipeline,
    package_path="ml_pipeline.yaml"
)

print("Pipeline compiled successfully!")
print("Generated: ml_pipeline.yaml")

```

8.3 requirements.txt

```

kfp==2.17.0
docker==7.2.0

```

8.4 verify_backend.sh

```

#!/usr/bin/env bash
set -e

echo "=== Kubernetes ==="
kubectl cluster-info
kubectl get nodes

echo
echo "=== KFP Pods ==="
kubectl get pods -n kubeflow

echo
echo "=== KFP Services ==="
kubectl get svc -n kubeflow

echo
echo "=== KFP Workflows ==="
kubectl get workflows -n kubeflow

```

8.5 start_kfp_ui.sh

```
#!/usr/bin/env bash
echo "Starting KFP Dashboard port-forward..."
echo "Keep this terminal open while using the UI."
kubectl port-forward -n kubeflow svc/ml-pipeline-ui 8080:80
```

Important about ml_pipeline.yaml: The real uploadable YAML is generated by KFP 2.17.0. It should not be fabricated or manually typed. Run `python compile_pipeline.py` in the same KFP environment used for the lab.

9. Hands-on Part A: Verify the Environment

Open Ubuntu/WSL and activate the project environment.

```
cd ~/kubeflow-mlops/ci-kfp-demo
source ../.venv/bin/activate
python --version
```

Verify Kubernetes:

```
kubectl cluster-info
kubectl get nodes
```

Expected: the control-plane should be reachable and the node should be Ready.

Verify KFP:

```
kubectl get pods -n kubeflow
```

Most KFP pods should show Running/Ready. If not, stop here and troubleshoot the first failing pod before continuing.

10. Hands-on Part B: Compile the Pipeline

```
cd ~/kubeflow-mlops/ci-kfp-demo
python compile_pipeline.py
```

Expected:

```
Pipeline compiled successfully!
Generated: ml_pipeline.yaml
```

Check the generated file:

```
ls -lh ml_pipeline.yaml
head -30 ml_pipeline.yaml
```

Concept: The YAML is the compiled pipeline specification. It is not the Python source code.

11. Hands-on Part C: Start KFP Dashboard

Open a dedicated WSL terminal and keep it running:

```
kubectl port-forward -n kubeflow svc/ml-pipeline-ui 8080:80
```

Expected:

```
Forwarding from 127.0.0.1:8080 -> 3000
Forwarding from [::1]:8080 -> 3000
```

Open: <http://localhost:8080>

Very important: The port-forward is only the local connection to the UI. If it stops, the browser may show 'failed to retrieve run' even when the backend itself is healthy.

12. Hands-on Part D: Upload the Pipeline

25. Open Pipelines.

26. Choose Upload pipeline.
27. Select ml_pipeline.yaml.
28. Use pipeline name: Iris ML Quality Gate Pipeline.
29. Verify the pipeline graph.
30. Confirm the uploaded pipeline version is visible.

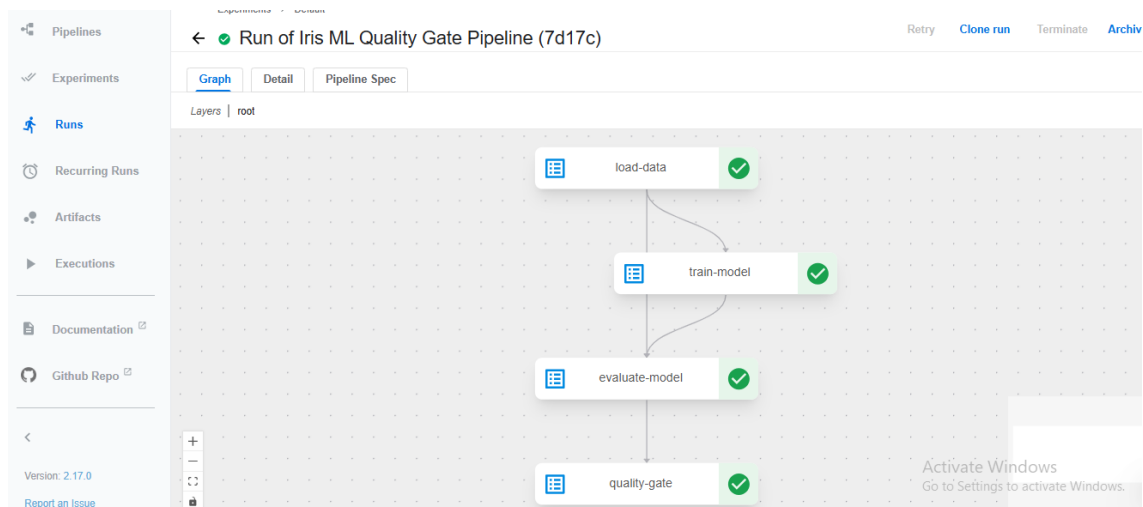
```
load-data → train-model → evaluate-model → quality-gate
```

13. Hands-on Part E: Manual Run

31. Open Runs.
32. Click Create run.
33. Select Iris ML Quality Gate Pipeline.
34. Select the uploaded pipeline version.
35. Use the Default experiment.
36. Name the run, for example Iris Manual Run 1.
37. Start the run.
38. Open the run and monitor each component.

```
load-data      ✓
train-model    ✓
evaluate-model ✓
quality-gate   ✓
Overall: Succeeded
```

Open component logs to see data counts, training messages, accuracy and quality-gate result.



14. Hands-on Part F: Create a Recurring Run

Open Recurring Runs and click + Create recurring run.

Field	Value used in this lab
Pipeline	Iris ML Quality Gate Pipeline
Pipeline Version	Successful uploaded version
Recurring run name	Iris 3-Minute Scheduled Run
Description	Automatically execute the Iris ML pipeline on a recurring schedule.

Experiment	Default
Run Type	Recurring
Trigger type	Periodic
Maximum concurrent runs	10
Has start date	Unchecked
Has end date	Unchecked
Catchup	Unchecked
Run every	3 Minutes
Custom Pipeline Root	Unchecked
Run parameters	None

Click Start only after checking all settings.

10

☐ Has start date

☐ Has end date

☐ Catchup ?

Run every

3

Minutes ▼

Pipeline Root

Pipeline Root represents an artifact repository, refer to [Pipeline Root Documentation](#).

☐ Custom Pipeline Root

Run parameters

This pipeline has no parameters

Start

Cancel

15. Hands-on Part G: Verify Automatic Runs

After creating the recurring run, open its details.

```

Enabled: Yes
Trigger: Every 3 minutes
Max. concurrent runs: 10
Catchup: false

```

Then open Runs and refresh after the next interval.

```

Example:
6:02:58 PM -> scheduled Run 1 -> Succeeded
6:05:59 PM -> scheduled Run 2 -> Running
next       -> scheduled Run 3
...

```

The completed lab showed this pattern: a previous scheduled run succeeded and a new run appeared about three minutes later.

Pipelines

Experiments

Runs

Recurring Runs

Artifacts

Executions

Documentation

Github Repo

← Run of Iris ML Quality Gate Pipeline (a8663)

[Clone recurring run](#)
[Refresh](#)
[Enable](#)
[Disable](#)
[Delete](#)

Recurring run details

Description

Created at

9/25/2026, 5:59:43 PM

Run trigger

Enabled

Yes

Trigger

Every 3 minutes

Max. concurrent runs

10

Catchup

false

Pipelines

Experiments

Runs

Recurring Runs

Artifacts

Executions

Documentation

Github Repo

Runs

[+ Create run](#)
[Compare runs](#)
[Clone run](#)
[Archive](#)
[Refresh](#)

Active

Archived

Filter runs

☐	Run name	Status	Duration	Experiment	Pipeline Version	Recurring Run	Start time ↓
☐	runofirismlqualitygatepipcms4j-2-1634...		-	Default	Iris ML Quality Gate Pipe...	[View config]	9/25/2026, 6:05:59 PM
☐	runofirismlqualitygatepipcms4j-1-1617...		0:02:13	Default	Iris ML Quality Gate Pipe...	[View config]	9/25/2026, 6:02:58 PM
☐	Run of Iris ML Quality Gate Pipeline (7...		0:17:13	Default	Iris ML Quality Gate Pipe...	-	9/25/2026, 12:27:22 PM

Rows per page: 10 < >

16. Detailed Explanation of Scheduling

One-off:

User -> Start -> One pipeline run

Recurring:

Recurring configuration

|

v

Periodic trigger

|

+--> Run 1

+--> Run 2

+--> Run 3

+--> ...

16.1 Periodic vs Cron

Trigger	Meaning	Example
Periodic	Fixed interval	Every 3 minutes
Cron	Calendar-style schedule	Specific time/day pattern

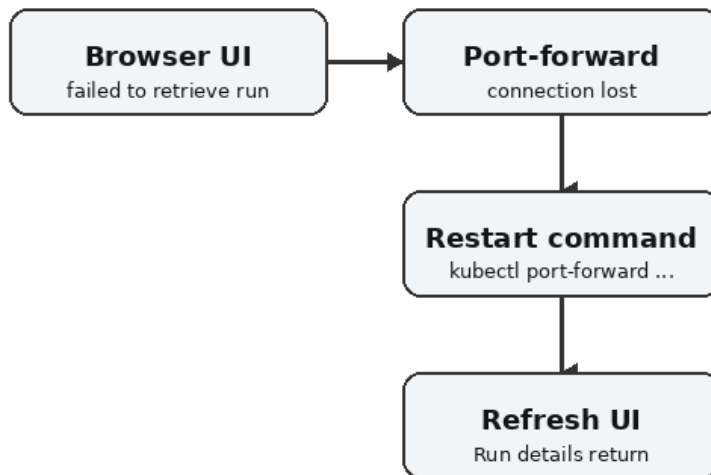
16.2 Catchup

Catchup controls how missed scheduled executions are treated after scheduler downtime. For this classroom demonstration, Catchup was disabled so that a backlog of missed demonstration runs would not be the focus.

16.3 Maximum concurrent runs

If a pipeline takes longer than the scheduling interval, multiple runs may overlap. Maximum concurrent runs places a limit on simultaneous executions for the recurring configuration.

17. Error Tracing and Troubleshooting



First restore the UI connection; do not reinstall KFP automatically.

17.1 UI error: 'Error: failed to retrieve run'

Observed during the lab after the system had been idle for several hours. The immediate cause was the local `kubectl port-forward` connection terminating.

```
kubectl port-forward -n kubeflow svc/ml-pipeline-ui 8080:80
```

Then refresh `http://localhost:8080`. Do not reinstall KFP just because the browser cannot retrieve a run; first restore the UI port-forward.

17.2 Terminal error: 'lost connection to pod'

This appeared in the port-forward terminal. It means the forwarding session lost its connection to the UI pod.

```
kubectl port-forward -n kubeflow svc/ml-pipeline-ui 8080:80
```

Restarting the port-forward restored the dashboard.

17.3 UI metadata-store warning

During an earlier manual-run attempt, the UI temporarily displayed a metadata-store retrieval error while the workflow still progressed. After the backend was cleanly rebuilt and the services became healthy, the manual run completed successfully. The lesson is to check actual workflow/pod status before assuming a transient UI banner means pipeline failure.

```
kubectl get pods -n kubeflow
kubectl get workflows -n kubeflow
```

17.4 Pipeline run fails

```
KFP Dashboard
|
+--> Runs
    |
    +--> failed run
        |
        +--> failed task
            |
            +--> Logs
```

Find the first failed task and inspect its logs before investigating downstream tasks.

17.5 Scheduler creates no runs

```
kubectl get pods -n kubeflow
kubectl logs deployment/ml-pipeline-scheduledworkflow -n kubeflow
kubectl logs deployment/workflow-controller -n kubeflow
```

Also check that the recurring configuration is Enabled = Yes and the trigger is configured as intended.

17.6 KFP pods show ImagePullBackOff

This is an image-pull problem, not a pipeline-code problem. Inspect the pod events:

```
kubectl get pods -n kubeflow
kubectl describe pod <pod-name> -n kubeflow
```

Look at the Events section for the actual image-pull message.

17.7 Recurring run is disabled

Open Recurring Runs, open the configuration, and use Enable. Verify Enabled = Yes afterward.

17.8 Too many overlapping runs

If execution takes longer than the schedule interval, runs can overlap. Review execution duration, schedule interval and maximum concurrent runs. The three-minute interval is intentionally short for classroom demonstration.

18. Observed Lab Errors and Their Resolution

Observed symptom	Diagnosis	Action
UI failed to retrieve run after ~3 hours	Port-forward session lost	Restart kubectl port-forward and refresh
Terminal: lost connection to pod	Port-forward connection ended	Restart port-forward
Transient metadata-store UI banner	Temporary UI/backend metadata interaction issue	Check pod/workflow status; do not assume pipeline failure
Scheduler client out-of-cluster kubeconfig warning	Backend tried out-of-cluster configuration first	In-cluster configuration followed; not a blocker
Workflow tasks completed successfully	Backend execution healthy	Verify green task status and logs
Recurring runs appeared every ~3 minutes	Scheduler working	Verify automatic executions in Runs

19. Practical Result and Evidence

The practical is successful when:

- Kubernetes node is Ready.
- KFP pods are healthy.
- ml_pipeline.py compiles successfully.
- ml_pipeline.yaml is generated.
- Pipeline is visible in the Dashboard.
- Manual run succeeds.
- Quality gate reports PASS.
- Recurring run shows Enabled = Yes.
- Trigger shows Every 3 minutes.
- New runs appear automatically.

The completed lab produced a successful manual run and scheduled runs at approximately three-minute intervals.

20. Viva Questions

Q. What is KFP?

A. A platform for defining and executing ML workflows.

Q. Why compile a pipeline?

A. To convert the Python pipeline definition into a specification the backend can execute.

Q. What is the KFP backend?

A. The server-side system that manages pipeline definitions, runs and orchestration functions.

Q. What is a pipeline run?

A. One execution of a pipeline.

Q. What is a recurring run?

A. A configuration that repeatedly creates pipeline runs according to a trigger.

Q. What is a Periodic trigger?

A. A fixed-interval trigger such as every 3 minutes.

Q. What is a Cron trigger?

A. A trigger based on Cron scheduling rules.

Q. Why Kubernetes?

A. It provides infrastructure for containerized and orchestrated task execution.

Q. Why Docker?

A. It packages code and dependencies into reproducible execution environments.

Q. What is Catchup?

A. A scheduling setting affecting missed scheduled executions.

Q. What is maximum concurrent runs?

A. A limit on simultaneous executions for the recurring configuration.

Q. Why did the UI fail after several hours?

A. The local port-forward session ended; the backend could still be running.

Q. How do you trace a failed pipeline?

A. Find the first failed task and inspect its logs, then inspect Kubernetes workflow/pod state if needed.

Q. Scheduling vs orchestration?

A. Scheduling determines when a workflow is triggered; orchestration coordinates tasks, dependencies and execution.

21. Student Submission Checklist

- ☐ kubectl get nodes screenshot
- ☐ kubectl get pods -n kubeflow screenshot
- ☐ Successful compilation screenshot
- ☐ Pipeline graph screenshot
- ☐ Successful manual run screenshot
- ☐ Evaluate-model log screenshot
- ☐ Recurring Run configuration screenshot
- ☐ Recurring Run details showing Enabled = Yes and Every 3 minutes
- ☐ Runs page showing at least one scheduled run
- ☐ Short explanation of local vs backend execution
- ☐ Short explanation of scheduling vs orchestration
- ☐ One troubleshooting example

22. Quick Command Sheet

```
# Activate environment
cd ~/kubeflow-mlops/ci-kfp-demo
source ../.venv/bin/activate

# Kubernetes
kubectl cluster-info
kubectl get nodes
kubectl get pods -n kubeflow
kubectl get svc -n kubeflow

# Compile
python compile_pipeline.py
ls -lh ml_pipeline.yaml

# Start KFP UI
kubectl port-forward -n kubeflow svc/ml-pipeline-ui 8080:80

# Workflows
kubectl get workflows -n kubeflow

# Scheduler troubleshooting
kubectl logs deployment/ml-pipeline-scheduledworkflow -n kubeflow
kubectl logs deployment/workflow-controller -n kubeflow

# Dashboard
http://localhost:8080
```

23. Related Files and How to Use Them

- `code/ml_pipeline.py` — complete pipeline source.
- `code/compile_pipeline.py` — generates the real `ml_pipeline.yaml`.
- `code/requirements.txt` — Python dependency versions.
- `code/verify_backend.sh` — backend verification commands.
- `code/start_kfp_ui.sh` — starts UI port-forward.
- `code/ml_pipeline.yaml`.README — explanation of generated YAML.
- `diagrams/` — architecture, pipeline/scheduler and error-tracing diagrams.
- `screenshots/` — selected evidence from the completed lab.
- `README.txt` — bundle overview.

Important: The bundle deliberately does not contain a fabricated `ml_pipeline.yaml`. The real YAML must be generated by KFP 2.17.0 from `ml_pipeline.py`. This avoids giving students a YAML file that looks valid but cannot be trusted as the exact compiled artifact.

24. Official References

- **Kubeflow Pipelines Overview:** <https://www.kubeflow.org/docs/components/pipelines/overview/>
- **KFP Installation:** <https://www.kubeflow.org/docs/components/pipelines/operator-guides/installation/>
- **Run a Pipeline:** <https://www.kubeflow.org/docs/components/pipelines/user-guides/core-functions/run-a-pipeline/>
- **Run and Recurring Run:** <https://www.kubeflow.org/docs/components/pipelines/concepts/run/>
- **Run Trigger:** <https://www.kubeflow.org/docs/components/pipelines/concepts/run-trigger/>
- **Compile a Pipeline:** <https://www.kubeflow.org/docs/components/pipelines/user-guides/core-functions/compile-a-pipeline/>

- **Execute KFP Pipelines Locally:** <https://www.kubeflow.org/docs/components/pipelines/user-guides/core-functions/execute-kfp-pipelines-locally/>